

Veo 3.1 is here! Read about the new model and its features in the [blog post](https://developers.googleblog.com/en/introducing-veo-3-1-and-new-creative-controls-in-the-gemini-api/)

(<https://developers.googleblog.com/en/introducing-veo-3-1-and-new-creative-controls-in-the-gemini-api/>)

and [documentation](https://ai.google.dev/gemini-api/docs/video) (<https://ai.google.dev/gemini-api/docs/video>).

Function calling with the Gemini API

Function calling lets you connect models to external tools and APIs. Instead of generating text responses, the model determines when to call specific functions and provides the necessary parameters to execute real-world actions. This allows the model to act as a bridge between natural language and real-world actions and data. Function calling has 3 primary use cases:

- **Augment Knowledge:** Access information from external sources like databases, APIs, and knowledge bases.
- **Extend Capabilities:** Use external tools to perform computations and extend the limitations of the model, such as using a calculator or creating charts.
- **Take Actions:** Interact with external systems using APIs, such as scheduling appointments, creating invoices, sending emails, or controlling smart home devices.

Get Weather

✓ Schedule Meeting

Create Chart

PythonJavaScript (#javascript)REST (#rest)
(#python)

```
from google import genai
from google.genai import types

# Define the function declaration for the model
schedule_meeting_function = {
    "name": "schedule_meeting",
    "description": "Schedules a meeting with specified attendees at a given time",
    "parameters": {
        "type": "object",
        "properties": {
            "attendees": {
                "type": "array",
                "items": {"type": "string"},
                "description": "List of people attending the meeting.",
            },
            "time": {
                "type": "string",
                "description": "The time to schedule the meeting.",
            },
        },
    },
}
```

```

        },
        "date": {
            "type": "string",
            "description": "Date of the meeting (e.g., '2024-07-29')",
        },
        "time": {
            "type": "string",
            "description": "Time of the meeting (e.g., '15:00')",
        },
        "topic": {
            "type": "string",
            "description": "The subject or topic of the meeting.",
        },
    },
    "required": ["attendees", "date", "time", "topic"],
},
}

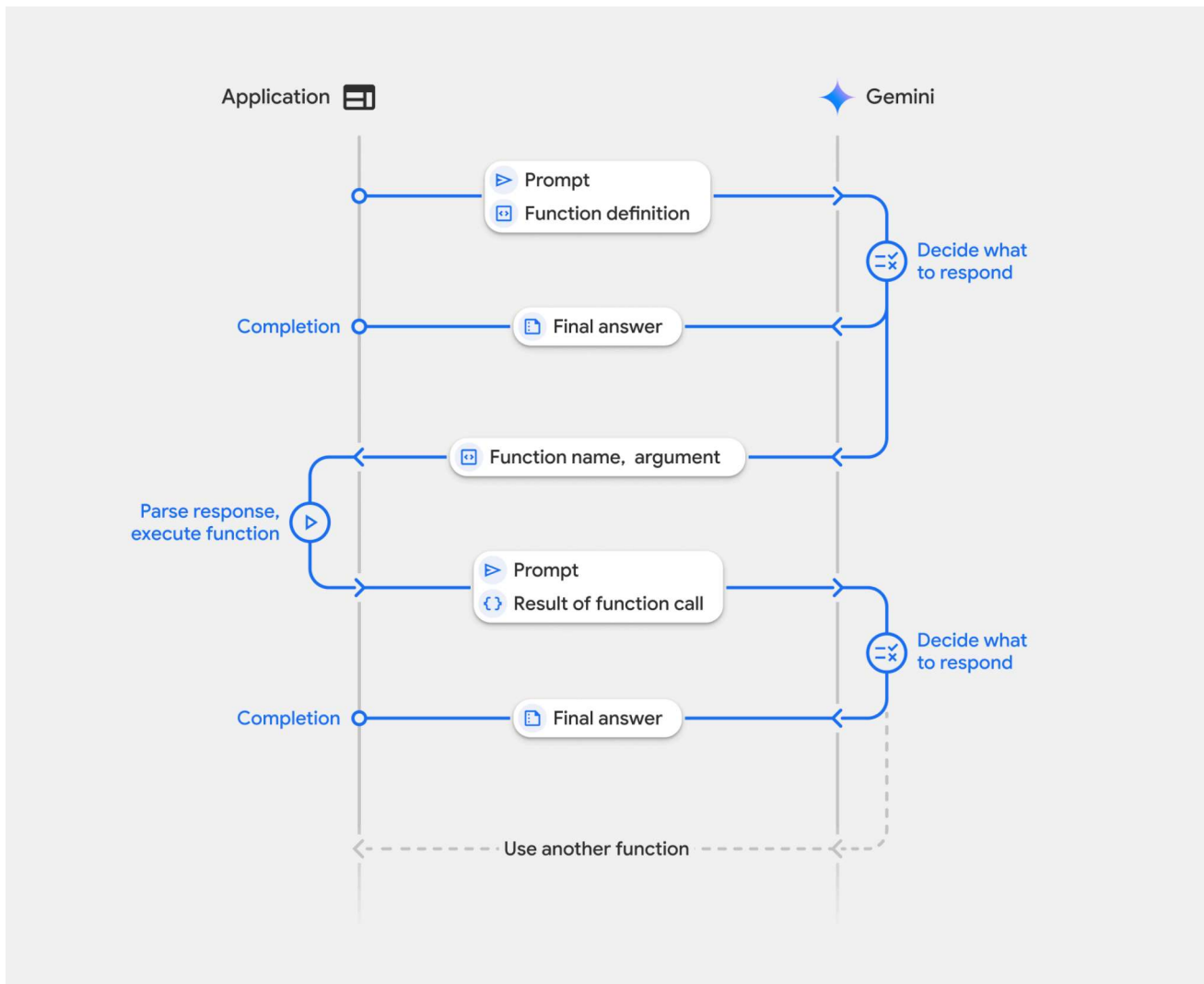
# Configure the client and tools
client = genai.Client()
tools = types.Tool(function_declarations=[schedule_meeting_function])
config = types.GenerateContentConfig(tools=[tools])

# Send request with function declarations
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="Schedule a meeting with Bob and Alice for 03/14/2025 at 10",
    config=config,
)

# Check for a function call
if response.candidates[0].content.parts[0].function_call:
    function_call = response.candidates[0].content.parts[0].function_call
    print(f"Function to call: {function_call.name}")
    print(f"Arguments: {function_call.args}")
    # In a real app, you would call your function here:
    # result = schedule_meeting(**function_call.args)
else:
    print("No function call found in the response.")
    print(response.text)

```

How function calling works



Function calling involves a structured interaction between your application, the model, and external functions. Here's a breakdown of the process:

1. **Define Function Declaration:** Define the function declaration in your application code. Function Declarations describe the function's name, parameters, and purpose to the model.
2. **Call LLM with function declarations:** Send user prompt along with the function declaration(s) to the model. It analyzes the request and determines if a function call would be helpful. If so, it responds with a structured JSON object.
3. **Execute Function Code (Your Responsibility):** The Model *does not* execute the function itself. It's your application's responsibility to process the response and check for Function Call, if
 - **Yes:** Extract the name and args of the function and execute the corresponding function in your application.
 - **No:** The model has provided a direct text response to the prompt (this flow is less emphasized in the example but is a possible outcome).

4. **Create User friendly response:** If a function was executed, capture the result and send it back to the model in a subsequent turn of the conversation. It will use the result to generate a final, user-friendly response that incorporates the information from the function call.

This process can be repeated over multiple turns, allowing for complex interactions and workflows. The model also supports calling multiple functions in a single turn ([parallel function calling](/gemini-api/docs/function-calling#parallel_function_calling) (/gemini-api/docs/function-calling#parallel_function_calling)) and in sequence ([compositional function calling](/gemini-api/docs/function-calling#compositional_function_calling) (/gemini-api/docs/function-calling#compositional_function_calling)).

Step 1: Define a function declaration

Define a function and its declaration within your application code that allows users to set light values and make an API request. This function could call external services or APIs.

PythonJavaScript (#javascript)
(#python)

```
# Define a function that the model can call to control smart lights
set_light_values_declaration = {
    "name": "set_light_values",
    "description": "Sets the brightness and color temperature of a light",
    "parameters": {
        "type": "object",
        "properties": {
            "brightness": {
                "type": "integer",
                "description": "Light level from 0 to 100. Zero is off a",
            },
            "color_temp": {
                "type": "string",
                "enum": ["daylight", "cool", "warm"],
                "description": "Color temperature of the light fixture, ",
            },
        },
        "required": ["brightness", "color_temp"],
    },
}

# This is the actual function that would be called based on the model's
def set_light_values(brightness: int, color_temp: str) -> dict[str, int]
    """Set the brightness and color temperature of a room light. (mock A

Args:
    brightness: Light level from 0 to 100. Zero is off and 100 is fu
```

```

        color_temp: Color temperature of the light fixture, which can be

Returns:
    A dictionary containing the set brightness and color temperature
    """
    return {"brightness": brightness, "colorTemperature": color_temp}

```

Step 2: Call the model with function declarations

Once you have defined your function declarations, you can prompt the model to use them. It analyzes the prompt and function declarations and decides whether to respond directly or to call a function. If a function is called, the response object will contain a function call suggestion.

PythonJavaScript (#javascript)
(#python)

```

from google.genai import types

# Configure the client and tools
client = genai.Client()
tools = types.Tool(function_declarations=[set_light_values_declaration])
config = types.GenerateContentConfig(tools=[tools])

# Define user prompt
contents = [
    types.Content(
        role="user", parts=[types.Part(text="Turn the lights down to a r
    )
]

# Send request with function declarations
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=contents
    config=config,
)

print(response.candidates[0].content.parts[0].function_call)

```

The model then returns a `functionCall` object in an OpenAPI compatible schema specifying how to call one or more of the declared functions in order to respond to the user's question.

PythonJavaScript (#javascript)
(#python)

```
id=None args={'color_temp': 'warm', 'brightness': 25} name='set_light_va
```

Step 3: Execute set_light_values function code

Extract the function call details from the model's response, parse the arguments , and execute the set_light_values function.

PythonJavaScript (#javascript)
(#python)

```
# Extract tool call details, it may not be in the first part.
tool_call = response.candidates[0].content.parts[0].function_call

if tool_call.name == "set_light_values":
    result = set_light_values(**tool_call.args)
    print(f"Function execution result: {result}")
```

Step 4: Create user friendly response with function result and call the model again

Finally, send the result of the function execution back to the model so it can incorporate this information into its final response to the user.

PythonJavaScript (#javascript)
(#python)

```
from google import genai
from google.genai import types

# Create a function response part
function_response_part = types.Part.from_function_response(
    name=tool_call.name,
    response={"result": result},
)

# Append function call and result of the function execution to contents
contents.append(response.candidates[0].content) # Append the content fro
```

```
contents.append(types.Content(role="user", parts=[function_response_part

client = genai.Client()
final_response = client.models.generate_content(
    model="gemini-2.5-flash",
    config=config,
    contents=contents,
)

print(final_response.text)
```

This completes the function calling flow. The model successfully used the `set_light_values` function to perform the request action of the user.

Function declarations

When you implement function calling in a prompt, you create a `tools` object, which contains one or more **function declarations**. You define functions using JSON, specifically with a [select subset](https://ai.google.dev/api/caching#Schema) (<https://ai.google.dev/api/caching#Schema>) of the [OpenAPI schema](https://spec.openapis.org/oas/v3.0.3#schema) (<https://spec.openapis.org/oas/v3.0.3#schema>) format. A single function declaration can include the following parameters:

- **name** (string): A unique name for the function (`get_weather_forecast`, `send_email`). Use descriptive names without spaces or special characters (use underscores or camelCase).
- **description** (string): A clear and detailed explanation of the function's purpose and capabilities. This is crucial for the model to understand when to use the function. Be specific and provide examples if helpful ("Finds theaters based on location and optionally movie title which is currently playing in theaters.").
- **parameters** (object): Defines the input parameters the function expects.
 - **type** (string): Specifies the overall data type, such as `object`.
 - **properties** (object): Lists individual parameters, each with:
 - **type** (string): The data type of the parameter, such as `string`, `integer`, `boolean`, `array`.
 - **description** (string): A description of the parameter's purpose and format. Provide examples and constraints ("The city and state, e.g., 'San Francisco, CA' or a zip code e.g., '95616'").

- **enum** (array, optional): If the parameter values are from a fixed set, use "enum" to list the allowed values instead of just describing them in the description. This improves accuracy ("enum": ["daylight", "cool", "warm"]).
- **required** (array): An array of strings listing the parameter names that are mandatory for the function to operate.

You can also construct `FunctionDeclarations` from Python functions directly using `types.FunctionDeclaration.from_callable(client=client, callable=your_function)`.

Function calling with thinking

Enabling "[thinking](/gemini-api/docs/thinking) (/gemini-api/docs/thinking)" can improve function call performance by allowing the model to reason through a request before suggesting function calls. The Gemini API is stateless, the model's reasoning context will be lost between turns in a multi-turn conversation. To preserve this context, you can use thought signatures. A thought signature is an encrypted representation of the model's internal thought process that you pass back to the model on subsequent turns.

The standard pattern for multi-turn tool

(/gemini-api/docs/function-calling?example=weather#step-4) use is to append the model's complete previous response to the conversation history. The `content` object includes the `thought_signatures` automatically. If you follow this pattern **No code changes are required**.

Manually managing thought signatures

If you modify the conversation history manually—instead of sending the complete previous response and want to benefit from thinking you must correctly handle the `thought_signature` included in the model's turn.

Follow these rules to ensure the model's context is preserved:

- Always send the `thought_signature` back to the model inside its original `Part`.
- Don't merge a `Part` containing a signature with one that does not. This breaks the positional context of the thought.
- Don't combine two `Parts` that both contain signatures, as the signature strings cannot be merged.

Inspecting Thought Signatures

While not necessary for implementation, you can inspect the response to see the `thought_signature` for debugging or educational purposes.

PythonJavaScript (#javascript)
(#python)

```
import base64
# After receiving a response from a model with thinking enabled
# response = client.models.generate_content(...)

# The signature is attached to the response part containing the function
part = response.candidates[0].content.parts[0]
if part.thought_signature:
    print(base64.b64encode(part.thought_signature).decode("utf-8"))
```

Learn more about limitations and usage of thought signatures, and about thinking models in general, on the [Thinking](/gemini-api/docs/thinking#signatures) (/gemini-api/docs/thinking#signatures) page.

Parallel function calling

In addition to single turn function calling, you can also call multiple functions at once. Parallel function calling lets you execute multiple functions at once and is used when the functions are not dependent on each other. This is useful in scenarios like gathering data from multiple independent sources, such as retrieving customer details from different databases or checking inventory levels across various warehouses or performing multiple actions such as converting your apartment into a disco.

PythonJavaScript (#javascript)
(#python)

```
power_disco_ball = {
    "name": "power_disco_ball",
    "description": "Powers the spinning disco ball.",
    "parameters": {
        "type": "object",
        "properties": {
            "power": {
                "type": "boolean",
                "description": "Whether to turn the disco ball on or off"
```

```

    }
    },
    "required": ["power"],
  },
}

start_music = {
  "name": "start_music",
  "description": "Play some music matching the specified parameters.",
  "parameters": {
    "type": "object",
    "properties": {
      "energetic": {
        "type": "boolean",
        "description": "Whether the music is energetic or not.",
      },
      "loud": {
        "type": "boolean",
        "description": "Whether the music is loud or not.",
      },
    },
    "required": ["energetic", "loud"],
  },
}

dim_lights = {
  "name": "dim_lights",
  "description": "Dim the lights.",
  "parameters": {
    "type": "object",
    "properties": {
      "brightness": {
        "type": "number",
        "description": "The brightness of the lights, 0.0 is off",
      },
    },
    "required": ["brightness"],
  },
}

```

Configure the function calling mode to allow using all of the specified tools. To learn more, you can read about [configuring function calling](https://ai.google.dev/gemini-api/docs/function-calling#function_calling_modes) (/gemini-api/docs/function-calling#function_calling_modes).

PythonJavaScript (#javascript)
(#python)

```

from google import genai
from google.genai import types

# Configure the client and tools
client = genai.Client()
house_tools = [
    types.Tool(function_declarations=[power_disco_ball, start_music, dim
]
config = types.GenerateContentConfig(
    tools=house_tools,
    automatic_function_calling=types.FunctionCallingConfig(
        disable=True
    ),
    # Force the model to call 'any' function, instead of chatting.
    tool_config=types.ToolConfig(
        function_calling_config=types.FunctionCallingConfig(mode='ANY')
    ),
)

chat = client.chats.create(model="gemini-2.5-flash", config=config)
response = chat.send_message("Turn this place into a party!")

# Print out each of the function calls requested from this single call
print("Example 1: Forced function calling")
for fn in response.function_calls:
    args = ", ".join(f"{key}={val}" for key, val in fn.args.items())
    print(f"{fn.name}({args})")

```

Each of the printed results reflects a single function call that the model has requested. To send the results back, include the responses in the same order as they were requested.

The Python SDK supports [automatic function calling](#)

([/gemini-api/docs/function-calling#automatic_function_calling_python_only](#)), which automatically converts Python functions to declarations, handles the function call execution and response cycle for you. Following is an example for the disco use case.

Note: Automatic Function Calling is a Python SDK only feature at the moment.

Python
(#python)

```

from google import genai
from google.genai import types

# Actual function implementations
def power_disco_ball_impl(power: bool) -> dict:
    """Powers the spinning disco ball.

    Args:
        power: Whether to turn the disco ball on or off.

    Returns:
        A status dictionary indicating the current state.
    """
    return {"status": f"Disco ball powered {'on' if power else 'off'}"}

def start_music_impl(energetic: bool, loud: bool) -> dict:
    """Play some music matching the specified parameters.

    Args:
        energetic: Whether the music is energetic or not.
        loud: Whether the music is loud or not.

    Returns:
        A dictionary containing the music settings.
    """
    music_type = "energetic" if energetic else "chill"
    volume = "loud" if loud else "quiet"
    return {"music_type": music_type, "volume": volume}

def dim_lights_impl(brightness: float) -> dict:
    """Dim the lights.

    Args:
        brightness: The brightness of the lights, 0.0 is off, 1.0 is full

    Returns:
        A dictionary containing the new brightness setting.
    """
    return {"brightness": brightness}

# Configure the client
client = genai.Client()
config = types.GenerateContentConfig(
    tools=[power_disco_ball_impl, start_music_impl, dim_lights_impl]
)

# Make the request

```

```

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="Do everything you need to this place into party!",
    config=config,
)

print("\nExample 2: Automatic function calling")
print(response.text)
# I've turned on the disco ball, started playing loud and energetic musi

```

Compositional function calling

Compositional or sequential function calling allows Gemini to chain multiple function calls together to fulfill a complex request. For example, to answer "Get the temperature in my current location", the Gemini API might first invoke a `get_current_location()` function followed by a `get_weather()` function that takes the location as a parameter.

The following example demonstrates how to implement compositional function calling using the Python SDK and automatic function calling.

PythonJavaScript (#javascript)
(#python)

This example uses the automatic function calling feature of the `google-genai` Python SDK. The SDK automatically converts the Python functions to the required schema, executes the function calls when requested by the model, and sends the results back to the model to complete the task.

```

import os
from google import genai
from google.genai import types

# Example Functions
def get_weather_forecast(location: str) -> dict:
    """Gets the current weather temperature for a given location."""
    print(f"Tool Call: get_weather_forecast(location={location})")
    # TODO: Make API call
    print("Tool Response: {'temperature': 25, 'unit': 'celsius'}")
    return {"temperature": 25, "unit": "celsius"} # Dummy response

def set_thermostat_temperature(temperature: int) -> dict:
    """Sets the thermostat to a desired temperature."""

```

```

    print(f"Tool Call: set_thermostat_temperature(temperature={temperatu
# TODO: Interact with a thermostat API
    print("Tool Response: {'status': 'success'}")
    return {"status": "success"}

# Configure the client and model
client = genai.Client()
config = types.GenerateContentConfig(
    tools=[get_weather_forecast, set_thermostat_temperature]
)

# Make the request
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="If it's warmer than 20°C in London, set the thermostat to :
    config=config,
)

# Print the final, user-facing response
print(response.text)

```

Expected Output

When you run the code, you will see the SDK orchestrating the function calls. The model first calls `get_weather_forecast`, receives the temperature, and then calls `set_thermostat_temperature` with the correct value based on the logic in the prompt.

```

Tool Call: get_weather_forecast(location=London)
Tool Response: {'temperature': 25, 'unit': 'celsius'}
Tool Call: set_thermostat_temperature(temperature=20)
Tool Response: {'status': 'success'}
OK. I've set the thermostat to 20°C.

```

Compositional function calling is a native [Live API](https://ai.google.dev/gemini-api/docs/live) (https://ai.google.dev/gemini-api/docs/live) feature. This means Live API can handle the function calling similar to the Python SDK.

PythonJavaScript (#javascript)
(#python)

```

# Light control schemas
turn_on_the_lights_schema = {'name': 'turn_on_the_lights'}
turn_off_the_lights_schema = {'name': 'turn_off_the_lights'}

```

```

prompt = """
Hey, can you write run some python code to turn on the lights, wait 10
"""

tools = [
    {'code_execution': {}},
    {'function_declarations': [turn_on_the_lights_schema, turn_off_the_l
]

await run(prompt, tools=tools, modality="AUDIO")

```

Function calling modes

The Gemini API lets you control how the model uses the provided tools (function declarations). Specifically, you can set the mode within the `function_calling_config`.

- **AUTO (Default):** The model decides whether to generate a natural language response or suggest a function call based on the prompt and context. This is the most flexible mode and recommended for most scenarios.
- **ANY:** The model is constrained to always predict a function call and guarantees function schema adherence. If `allowed_function_names` is not specified, the model can choose from any of the provided function declarations. If `allowed_function_names` is provided as a list, the model can only choose from the functions in that list. Use this mode when you require a function call response to every prompt (if applicable).
- **NONE:** The model is *prohibited* from making function calls. This is equivalent to sending a request without any function declarations. Use this to temporarily disable function calling without removing your tool definitions.

PythonJavaScript (#javascript)
(#python)

```

from google.genai import types

# Configure function calling mode
tool_config = types.ToolConfig(
    function_calling_config=types.FunctionCallingConfig(
        mode="ANY", allowed_function_names=["get_current_temperature"]
    )
)

```

```
)

# Create the generation config
config = types.GenerateContentConfig(
    tools=[tools], # not defined here.
    tool_config=tool_config,
)
```

Automatic function calling (Python only)

When using the Python SDK, you can provide Python functions directly as tools. The SDK converts these functions into declarations, manages the function call execution, and handles the response cycle for you. Define your function with type hints and a docstring. For optimal results, it is recommended to use [Google-style docstrings](https://google.github.io/styleguide/pyguide.html#383-functions-and-methods).

(<https://google.github.io/styleguide/pyguide.html#383-functions-and-methods>) The SDK will then automatically:

1. Detect function call responses from the model.
2. Call the corresponding Python function in your code.
3. Send the function's response back to the model.
4. Return the model's final text response.

The SDK currently does not parse argument descriptions into the property description slots of the generated function declaration. Instead, it sends the entire docstring as the top-level function description.

Python

(#python)

```
from google import genai
from google.genai import types

# Define the function with type hints and docstring
def get_current_temperature(location: str) -> dict:
    """Gets the current temperature for a given location.

    Args:
        location: The city and state, e.g. San Francisco, CA

    Returns:
```



```

        A dictionary containing the temperature and unit.
        """
        # ... (implementation) ...
        return {"temperature": 25, "unit": "Celsius"}

# Configure the client
client = genai.Client()
config = types.GenerateContentConfig(
    tools=[get_current_temperature]
) # Pass the function itself

# Make the request
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents="What's the temperature in Boston?",
    config=config,
)

print(response.text) # The SDK handles the function call and returns th

```

You can disable automatic function calling with:

Python

(#python)

```

config = types.GenerateContentConfig(
    tools=[get_current_temperature],
    automatic_function_calling=typesAutomaticFunctionCallingConfig(disable=True)
)

```

Automatic function schema declaration

The API is able to describe any of the following types. Pydantic types are allowed, as long as the fields defined on them are also composed of allowed types. Dict types (like `dict[str: int]`) are not well supported here, don't use them.

Python

(#python)

```

AllowedType = (
    int | float | bool | str | list['AllowedType'] | pydantic.BaseModel)

```

To see what the inferred schema looks like, you can convert it using [from_callable](https://googleapis.github.io/python-genai/genai.html#genai.types.FunctionDeclaration.from_callable) (https://googleapis.github.io/python-genai/genai.html#genai.types.FunctionDeclaration.from_callable):

Python

(#python)

```
from google import genai
from google.genai import types

def multiply(a: float, b: float):
    """Returns a * b."""
    return a * b

client = genai.Client()
fn_decl = types.FunctionDeclaration.from_callable(callable=multiply, cli

# to_json_dict() provides a clean JSON representation.
print(fn_decl.to_json_dict())
```

Multi-tool use: Combine native tools with function calling

You can enable multiple tools combining native tools with function calling at the same time. Here's an example that enables two tools, [Grounding with Google Search](/gemini-api/docs/grounding) (</gemini-api/docs/grounding>) and [code execution](/gemini-api/docs/code-execution) (</gemini-api/docs/code-execution>), in a request using the [Live API](/gemini-api/docs/live) (</gemini-api/docs/live>).

Note: Multi-tool use is a [Live API](https://ai.google.dev/gemini-api/docs/live) (<https://ai.google.dev/gemini-api/docs/live>) only feature at the moment. The `run()` function declaration, which handles the asynchronous websocket setup, is omitted for brevity.

PythonJavaScript (#javascript)

(#python)

```
# Multiple tasks example - combining lights, code execution, and search
prompt = """
Hey, I need you to do three things for me.

1. Turn on the lights.
2. Then compute the largest prime palindrome under 100000.
3. Then use Google Search to look up information about the largest
```

```

    Thanks!
    """

tools = [
    {'google_search': {}},
    {'code_execution': {}},
    {'function_declarations': [turn_on_the_lights_schema, turn_off_the_l
]

# Execute the prompt with specified tools in audio modality
await run(prompt, tools=tools, modality="AUDIO")

```

Python developers can try this out in the [Live API Tool Use notebook](https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_LiveAPI_tools.ipynb)
 (https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_LiveAPI_tools.ipynb)

Model context protocol (MCP)

[Model Context Protocol \(MCP\)](https://modelcontextprotocol.io/introduction) (https://modelcontextprotocol.io/introduction) is an open standard for connecting AI applications with external tools and data. MCP provides a common protocol for models to access context, such as functions (tools), data sources (resources), or predefined prompts.

The Gemini SDKs have built-in support for the MCP, reducing boilerplate code and offering [automatic tool calling](/gemini-api/docs/function-calling#automatic_function_calling_python_only) (/gemini-api/docs/function-calling#automatic_function_calling_python_only) for MCP tools. When the model generates an MCP tool call, the Python and JavaScript client SDK can automatically execute the MCP tool and send the response back to the model in a subsequent request, continuing this loop until no more tool calls are made by the model.

Here, you can find an example of how to use a local MCP server with Gemini and mcp SDK.

PythonJavaScript (#javascript)
 (#python)

Make sure the latest version of the [mcp SDK](https://modelcontextprotocol.io/introduction) (https://modelcontextprotocol.io/introduction) is installed on your platform of choice.

```
pip install mcp
```

✦ **Note:** Python supports automatic tool calling by passing in the `ClientSession` into the `tools` parameters. If you want to disable it, you can provide `automatic_function_calling` with disabled `True`.

```
import os
import asyncio
from datetime import datetime
from mcp import ClientSession, StdioServerParameters
from mcp.client.stdio import stdio_client
from google import genai

client = genai.Client()

# Create server parameters for stdio connection
server_params = StdioServerParameters(
    command="npx", # Executable
    args=["-y", "@philschmid/weather-mcp"], # MCP Server
    env=None, # Optional environment variables
)

async def run():
    async with stdio_client(server_params) as (read, write):
        async with ClientSession(read, write) as session:
            # Prompt to get the weather for the current day in London.
            prompt = f"What is the weather in London in {datetime.now()}."

            # Initialize the connection between client and server
            await session.initialize()

            # Send request to the model with MCP function declarations
            response = await client.ai.models.generate_content(
                model="gemini-2.5-flash",
                contents=prompt,
                config=genai.types.GenerateContentConfig(
                    temperature=0,
                    tools=[session], # uses the session, will automatic
                    # Uncomment if you **don't** want the SDK to automat
                    # automatic_function_calling=genai.types.AutomaticFu
                    #     disable=True
                    # ),
                ),
            )
            print(response.text)

# Start the asyncio event loop and run the main function
```

```
asyncio.run(run())
```

Limitations with built-in MCP support

Built-in MCP support is a experimental (/gemini-api/docs/models#preview) feature in our SDKs and has the following limitations:

- Only tools are supported, not resources nor prompts
- It is available for the Python and JavaScript/TypeScript SDK.
- Breaking changes might occur in future releases.

Manual integration of MCP servers is always an option if these limit what you're building.

Supported models

This section lists models and their function calling capabilities. Experimental models are not included. You can find a comprehensive capabilities overview on the model overview (https://ai.google.dev/gemini-api/docs/models) page.

Model	Function Calling	Parallel Function Calling	Compositional Function Calling
Gemini 2.5 Pro	✓	✓	✓
Gemini 2.5 Flash	✓	✓	✓
Gemini 2.5 Flash-Lite	✓	✓	✓
Gemini 2.0 Flash	✓	✓	✓
Gemini 2.0 Flash-Lite	X	X	X

Best practices

- **Function and Parameter Descriptions:** Be extremely clear and specific in your descriptions. The model relies on these to choose the correct function and provide appropriate arguments.
- **Naming:** Use descriptive function names (without spaces, periods, or dashes).
- **Strong Typing:** Use specific types (integer, string, enum) for parameters to reduce errors. If a parameter has a limited set of valid values, use an enum.
- **Tool Selection:** While the model can use an arbitrary number of tools, providing too many can increase the risk of selecting an incorrect or suboptimal tool. For best results, aim to provide only the relevant tools for the context or task, ideally keeping the active set to a maximum of 10-20. Consider dynamic tool selection based on conversation context if you have a large total number of tools.
- **Prompt Engineering:**
 - Provide context: Tell the model its role (e.g., "You are a helpful weather assistant.").
 - Give instructions: Specify how and when to use functions (e.g., "Don't guess dates; always use a future date for forecasts.").
 - Encourage clarification: Instruct the model to ask clarifying questions if needed.
- **Temperature:** Use a low temperature (e.g., 0) for more deterministic and reliable function calls.
- **Validation:** If a function call has significant consequences (e.g., placing an order), validate the call with the user before executing it.
- **Check Finish Reason:** Always check the [finishReason](#) (`/api/generate-content#FinishReason`) in the model's response to handle cases where the model failed to generate a valid function call.
- **Error Handling:** Implement robust error handling in your functions to gracefully handle unexpected inputs or API failures. Return informative error messages that the model can use to generate helpful responses to the user.
- **Security:** Be mindful of security when calling external APIs. Use appropriate authentication and authorization mechanisms. Avoid exposing sensitive data in function calls.
- **Token Limits:** Function descriptions and parameters count towards your input token limit. If you're hitting token limits, consider limiting the number of functions or the length of the descriptions, break down complex tasks into smaller, more focused function sets.

Notes and limitations

- Only a subset of the OpenAPI schema (/api/caching#FunctionDeclaration) is supported.
- Supported parameter types in Python are limited.
- Automatic function calling is a Python SDK feature only.

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2025-10-15 UTC.